

Arrays in C++ From One Block of Memory to Placement-Ready

Everything about arrays and vectors: what the memory actually looks like, the index discipline that kills off-by-one bugs, two pointers, sliding windows, prefix sums, binary search, hashing tricks, matrix manipulation, the 14 problem patterns, and a curated problem list.

69

SECTIONS

14

PATTERNS

56

CODE BLOCKS

38

PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Generated 14 September 2026

Contents

PART 0 — Arrays, from absolute zero

- 0.1 What the memory actually looks like
- 0.2 Why $a[i]$ is $O(1)$ — the one formula
- 0.3 The price of that superpower
- 0.4 The three dangerous facts about raw C arrays
- 0.5 The vocabulary
- 0.6 How to recognise the topic in a problem statement

PART 1 — The three array types in C++, and the STL toolkit

- 1.1 The three types side by side
- 1.2 Creating and filling a vector
- 1.3 Size, capacity, and why `push_back` is $O(1)$
- 1.4 `size()` is unsigned — the loop that runs forever
- 1.5 The STL toolkit you actually use
- 1.6 Iterating
- 1.7 2D arrays and matrices
- 1.8 A string is an array of characters
- 1.9 Complexity, all in one place

PART 2 — The index mental model

- 2.1 Every index must have a sentence
- 2.2 Half-open ranges: $[l, r)$
- 2.3 The four loop skeletons, and when each is right
- 2.4 The five inputs you dry-run, every single time
- 2.5 Overflow: the silent wrong answer
- 2.6 In-place means $O(1)$ extra space

PART 3 — Two pointers

- 3.1 Shape A — closing in from both ends
- 3.2 Shape B — the write pointer
- 3.3 Shape C — fast & slow, and partitioning
- 3.4 Choosing the shape — the decision line

PART 4 — The sliding window

- 4.1 Fixed window of size k
- 4.2 Variable window — the universal template
- 4.3 Windows over characters — the frequency array
- 4.4 The "at most K " trick — counting subarrays
- 4.5 Window checklist

PART 5 — Prefix sums and difference arrays

- 5.1 The construction, and the one formula
- 5.2 Prefix + hash map — the pattern windows cannot do
- 5.3 Prefix products, and the no-division rule
- 5.4 The difference array — range updates in $O(1)$
- 5.5 2D prefix sums

PART 6 — Binary search on arrays

- 6.1 Template 1 — find an exact value
- 6.2 Template 2 — find a boundary (the one that actually matters)
- 6.3 Rotated sorted arrays
- 6.4 Binary search on the answer
- 6.5 LC 4 — median of two sorted arrays

PART 7 — Hashing, frequency arrays, and using the array as its own hash table

- 7.1 The frequency array — the fastest hash table there is
- 7.2 `unordered_map` and `unordered_set`
- 7.3 Index-as-hash — when the values are $1..n$
- 7.4 XOR — the parity trick
- 7.5 Sorting as a preprocessing step

PART 8 — Kadane, intervals, and matrix manipulation

- 8.1 Kadane's algorithm
- 8.2 Intervals
- 8.3 Matrix manipulation
- 8.4 The rotation trick — three reversals
- 8.5 Boyer-Moore majority vote

PART 9 — The complete pattern taxonomy

Telling patterns 6 and 8 apart — the decision that matters most

Telling patterns 3 and 4 apart

PART 10 — How to approach ANY array problem

Step 1 — Read the constraints before the statement

Step 2 — State the brute force, with its complexity

Step 3 — Match a pattern from the table

Step 4 — Name every index before you write the loop

Step 5 — Write the guards first, then the body

Step 6 — Dry-run the five inputs

PART 11 — The bug catalogue

PART 12 — Complexity cheat sheet

By pattern

The brute force you are replacing

Space you might be spending without noticing

PART 13 — The problem list

Tier 1 — foundations (13 problems)

Tier 2 — the core patterns (15 problems)

Tier 3 — the ones that separate candidates (10 problems)

The Indian-placement classics

PART 14 — A four-week plan

Week 1 — foundations and the index discipline

Week 2 — two pointers and windows

Week 3 — prefix sums, binary search, hashing

Week 4 — the hard tier and revision

Quick reference card

PART 0 — Arrays, from absolute zero

An array is the very first data structure, and it is the one every other data structure is secretly built on. A vector is an array. A hash table is an array. A heap is an array. A stack is an array. So this topic is not "the easy one you skip" — it is the foundation, and roughly **half of every placement test you will sit** is array problems wearing different hats.

The whole idea is one sentence:

An array is one unbroken block of memory holding N values of the same type, so the address of element `i` can be computed instead of searched for.

Everything in this guide follows from that sentence. Let us take it apart.

0.1 What the memory actually looks like

Your program's memory is a very long row of numbered boxes. Each box holds one byte, and the number of a box is its **address**. When you write

```
int a[5] = {10, 20, 30, 40, 50};
```

the compiler reserves `5 * sizeof(int)` = 20 consecutive bytes and lays your values out in them, back to back, with nothing in between:

value	10	20	30	40	50
	a[0]	a[1]	a[2]	a[3]	a[4]
address	1000	1004	1008	1012	1016
	↑				
a	(the array starts here)				

`int` is 4 bytes on every machine you will meet in a contest, so the addresses step by 4.

0.2 Why `a[i]` is $O(1)$ — the one formula

To find element `i`, the machine does not walk through the array. It does arithmetic:

```
address of a[i] = address of a[0] + i * sizeof(element)
```

`a[3]` is at `1000 + 3*4 = 1012`. One multiply, one add, one memory read — the same amount of work whether `i` is 0 or 999999. **That constant-time random access is the array's superpower, and it is the reason to choose an array over a linked list.**

It also explains why indexing starts at 0: the index is not "which element" but "**how far from the start**". `a[0]` is zero steps from the beginning. Once you read `i` as a *distance*, half of your off-by-one confusion disappears.

Two consequences that get asked in vivas:

- `a[i]` is literally defined as `*(a + i)`. Since addition commutes, `a[i] == i[a]`. Try `3[a]` in a compiler — it compiles and works. Never write it; do know why it works.
- **All elements must be the same type.** The formula needs a single fixed `sizeof(element)`. That is why C++ arrays are homogeneous while a Python list is not.

0.3 The price of that superpower

An array buys $O(1)$ access by demanding one unbroken block. That has costs, and every array problem is really about dodging one of them:

OPERATION	COST	WHY
Read/write <code>a[i]</code>	$O(1)$	One address computation
Scan all elements	$O(n)$	Must touch each
Insert/erase at the end	$O(1)$ amortised	Nothing has to move
Insert/erase in the middle	$O(n)$	Everything after it shifts by one slot
Insert/erase at the front	$O(n)$	Worst case — everything shifts
Search in an unsorted array	$O(n)$	No structure to exploit
Search in a sorted array	$O(\log n)$	Binary search

```
erase a[1] from [10][20][30][40][50]
               ↓ every later element shifts left one slot
               [10][30][40][50][ ? ]      <-  $O(n)$  work
```

The trade in one line: a linked list gives cheap insertion and expensive access; an array gives cheap access and expensive insertion. Almost every interview problem wants access, which is why almost every interview problem is an array problem.

0.4 The three dangerous facts about raw C arrays

1. The size is fixed at compile time and is not stored anywhere. An array does not know how long it is. `int a[5];` — the `5` exists only in the type. This is why C functions always take a length alongside the pointer.

2. Arrays "decay" to pointers when passed to a function. This is the single biggest beginner trap:

```
void f(int arr[]) {           // this is a LIE – it really means int* arr
    cout << sizeof(arr);      // prints 8 (the size of a POINTER), not 20
}

int main() {
    int a[5] = {1,2,3,4,5};
    cout << sizeof(a);        // 20 – correct, here a is still an array
    cout << sizeof(a)/sizeof(a[0]); // 5 – the classic length idiom
    f(a);                    // only the address of a[0] is passed
}
```

Inside `f`, the length is gone forever. The array is **not copied** — you get the address of the original, so writes inside `f` change the caller's array. That is the C++ default for arrays and it surprises everyone once.

3. Going out of bounds is undefined behaviour, not an error. `a[7]` on a 5-element array does not crash and does not throw. It computes `a + 7*4` and reads whatever is sitting there — another variable, a return address, garbage. Sometimes your program works anyway, which is worse, because the bug shows up on the judge's hidden test case instead of yours.

```
int a[5];
a[5] = 99;    // silently corrupts whatever lives after the array
```

Use `std::vector` and index only inside `[0, n)`. During debugging you can use `v.at(i)`, which does throw `std::out_of_range`. In the final submission use `v[i]` — `at()` is slower because of the check.

0.5 The vocabulary

WORD	MEANING
Element	One value in the array
Index	How far an element is from the start; valid range <code>0 ... n-1</code>
Length / size	How many elements there are (<code>n</code>)
Capacity	How many a <code>vector</code> could hold before it must reallocate ($\geq$ size)
Subarray	A contiguous slice, <code>a[i..j]</code> . <code>[2,3]</code> of <code>[1,2,3,4]</code> — yes. <code>[1,3]</code> — no
Subsequence	Keeps order but may skip. <code>[1,3]</code> of <code>[1,2,3,4]</code> — yes
Subset	Any selection, order irrelevant
In-place	Using $O(1)$ extra memory; rearranging the array you were given
Stable	An algorithm that keeps equal elements in their original relative order

That subarray-versus-subsequence distinction is worth ten marks. **Contiguous** is the magic word: if a problem says "subarray" or "contiguous", sliding window and prefix sums are live options. If it says "subsequence", they are not — that is usually dynamic programming.

`n` contiguous subarrays: there are $n*(n+1)/2$ of them. `n` subsequences: 2^n . Brute forcing the first is sometimes acceptable; brute forcing the second never is.

0.6 How to recognise the topic in a problem statement

You are almost certainly in array-pattern territory when you see:

- "given an array / list of integers ..." and the answer is a number, an index, or the same array rearranged

- **"contiguous"**, "subarray", "window of size k", "consecutive"
- "sorted array" — binary search or two pointers, always
- "in-place", "O(1) extra space", "do not use another array"
- "find the pair / triplet that ..."
- "matrix" / "grid" with rotation, spiral, or zeroing
- "return the indices" rather than the values — that usually rules out sorting

And the sizes tell you the intended complexity before you have read the statement twice:

N UP TO	YOU NEED ABOUT
10	$O(n!)$ — permutations are fine
20	$O(2^n)$ — subsets
500	$O(n^3)$
5,000	$O(n^2)$
10^6	$O(n \log n)$ or $O(n)$
10^8	$O(n)$ with a small constant, or $O(\log n)$

Read the constraints first. If `$n \leq 10^5$` and you are planning a nested loop, stop planning and find the pattern. This one habit is worth more than any single algorithm in this guide.

PART 1 — The three array types in C++, and the STL toolkit

C++ gives you three things that are all "an array". Knowing which to use, and what each costs, is examinable and practical.

1.1 The three types side by side

```
int      a[5];           // C array – fixed size, no bounds check, no size()
array<int,5> b;          // std::array – same memory, but a real object
vector<int> c;           // std::vector – grows at runtime. Use this.
```

	INT A[N]	STD::ARRAY<INT,N>	STD::VECTOR<INT>
Size known at	compile time	compile time	runtime
Can grow	no	no	yes
Knows its size	no	.size()	.size()
Lives where	stack	stack	heap (the data)
Copies on assignment	no (decays)	yes	yes
Bounds-checked option	no	.at()	.at()

Rule for placements: use `vector<int>` for everything. It behaves like an array, the data is still one contiguous block so `v[i]` is still $O(1)$, and it removes the two worst C-array traps. Use a C array only for a small fixed lookup table, e.g. `int cnt[26] = {0};`.

1.2 Creating and filling a vector

```
vector<int> v;           // empty, size 0
vector<int> v(n);         // n elements, all 0
vector<int> v(n, -1);     // n elements, all -1
vector<int> v = {3, 1, 4, 1, 5}; // from a list
vector<int> w = v;        // a real, independent COPY (O(n))
vector<int> w(v.begin()+1, v.end()); // a copy of a slice
vector<vector<int>> g(r, vector<int>(c, 0)); // r x c grid of zeros
```

`vector<int> w = v;` copying is the opposite of the C-array behaviour and catches people going the other way. **If you pass a vector to a function by value you copy the whole thing, $O(n)$ every call.** Pass `const vector<int>& v` to read it and `vector<int>& v` to modify it.

```
int sum(vector<int> v);           // BAD – copies n elements per call
int sum(const vector<int>& v);     // GOOD – no copy, cannot modify
void sortIt(vector<int>& v);      // GOOD – no copy, can modify
```

1.3 Size, capacity, and why `push_back` is $O(1)$

A vector holds a pointer to a heap block, a **size** (how many you have) and a **capacity** (how many fit before it must move house).

```
size = 3, capacity = 4
```

10	20	30	..
----	----	----	----

`push_back(40)` -> just writes into the spare slot, $O(1)$

`push_back(50)` -> full. Allocate a block of 8, COPY all 4 across, free the old. $O(n)$.

When it is full, the vector allocates a **bigger block (typically double)**, copies everything, and frees the old one — that single `push_back` costs $O(n)$. But doubling means that expensive step happens on pushes 1, 2, 4, 8, 16 ..., so n pushes cost $1+2+4+\dots+n < 2n$ copies in total: **$O(1)$ amortised** per push. This is the same amortised argument as the monotonic stack, and interviewers do ask you to state it.

Practical consequences:

```
v.reserve(n);    // pre-allocate capacity for n – avoids all the reallocation. Use it in loops.
v.size();        // number of elements
v.capacity();    // never needed in an answer, sometimes asked in a viva
v.clear();       // size -> 0, capacity UNCHANGED (memory is kept)
v.shrink_to_fit(); // ask for the memory back
```

The dangling-reference bug: `push_back` may reallocate, which **invalidates every pointer, reference and iterator into the vector**. So `int& first = v[0]; v.push_back(9); first = 5;` can write into freed memory. Never hold a reference across a `push_back`. Hold an *index* instead — indices survive reallocation.

1.4 `size()` is unsigned — the loop that runs forever

```
for (int i = 0; i < v.size(); i++)    // warning: signed/unsigned comparison
for (int i = v.size() - 1; i >= 0; i--) // fine, i is int
```

`v.size()` returns `size_t`, an **unsigned** type. Two real bugs come from this:

```
// BUG 1 – empty vector, infinite loop:
for (size_t i = 0; i < v.size() - 1; i++) // v.size()==0 -> 0-1 = 18446744073709551615
// BUG 2 – comparing with a negative:
if (v.size() - 1 < 0)    // never true, ever
```

Fix: write `int n = v.size();` on the first line of every function and use `n` from then on. Every solution in the companion book starts that way.

1.5 The STL toolkit you actually use

```
#include <bits/stdc++.h>           // one header, allowed in contests and most judges
using namespace std;

sort(v.begin(), v.end());           // O(n log n), introsort
sort(v.begin(), v.end(), greater<int>()); // descending
sort(v.begin(), v.end(), [](const vector<int>& a, const vector<int>& b){
    return a[0] < b[0];             // custom: sort intervals by start
});
reverse(v.begin(), v.end());
int mx = *max_element(v.begin(), v.end());
int mn = *min_element(v.begin(), v.end());
long long s = accumulate(v.begin(), v.end(), 0LL); // NOTE the 0LL
int cnt = count(v.begin(), v.end(), 7);
bool found = binary_search(v.begin(), v.end(), 7); // sorted only
auto it = lower_bound(v.begin(), v.end(), 7);      // first >= 7
auto it2 = upper_bound(v.begin(), v.end(), 7);      // first > 7
int idx = lower_bound(v.begin(), v.end(), 7) - v.begin(); // iterator -> index
v.erase(unique(v.begin(), v.end(), v.end()), v.end()); // dedupe a SORTED vector
swap(v[i], v[j]);
next_permutation(v.begin(), v.end());
fill(v.begin(), v.end(), 0);
```

Three of those hide real traps:

- `accumulate(v.begin(), v.end(), 0)` **overflows**. The accumulator's type comes from the third argument, so `0` makes it an `int` and 10^5 elements of 10^4 each will wrap. Write `0LL`.
- `unique` **does not remove anything**. It shuffles the distinct values to the front and returns the new logical end; you must call `erase` with it, and the vector must be sorted first.
- `max_element` **returns an iterator**. Dereference it, and never dereference it on an empty vector.

1.6 Iterating

```
for (int i = 0; i < n; i++) cout << v[i]; // index – use when you NEED i
for (int x : v)             cout << x;    // read-only copy of each element
for (int& x : v)            x *= 2;       // modify in place
for (const string& s : names) cout << s; // avoid copying big elements
```

Prefer the index loop in interview answers: nearly every array pattern is about *where* you are, not just *what* you see. And **never modify a vector's size while range-for is iterating it** — the loop holds iterators and they will be invalidated.

1.7 2D arrays and matrices

```
vector<vector<int>> g(rows, vector<int>(cols, 0));
int r = g.size(), c = g[0].size();           // guard: g could be empty
g[i][j] = 5;                                 // row i, column j – ROW first
for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
        cout << g[i][j] << ' ';
```

	j=0	j=1	j=2	
i=0	1	2	3	<code>g[0][2] == 3</code>
i=1	4	5	6	<code>g[1][0] == 4</code>
i=2	7	8	9	<code>row = i (down), col = j (across)</code>

Two facts to keep:

- `vector<vector<int>>` is **not** one contiguous block — it is a vector of pointers to separate row blocks. A true 2D C array `int g[3][3]` is contiguous, laid out row after row (**row-major order**), so `g[i][j]` sits at `i*cols + j` from the start.
- That formula is how you **flatten** a grid into a single vector, which is often cleaner and always faster: `flat[i*cols + j]`. Going back: `i = idx / cols`, `j = idx % cols`. Worth memorising — it appears in matrix problems, in hashing, and in heaps.

The four-neighbour walk, which you will write a hundred times:

```
int dr[] = {-1, 1, 0, 0};
int dc[] = { 0, 0,-1, 1};
for (int d = 0; d < 4; d++) {
    int ni = i + dr[d], nj = j + dc[d];
    if (ni < 0 || ni >= r || nj < 0 || nj >= c) continue; // bounds FIRST
    // ... use g[ni][nj]
}
```

1.8 A string is an array of characters

Half of all "array" problems arrive disguised as strings. Everything transfers.

```
string s = "hello";
s[0]; s.size(); s.substr(1, 3); // (start, LENGTH) – not (start, end)
s += 'x'; s.push_back('x');
sort(s.begin(), s.end());
int idx = s[i] - 'a'; // 0..25, the key to frequency arrays
s[i] = 'A' + idx;
reverse(s.begin(), s.end());
stoi(s); to_string(42);
```

`s.substr(pos, len)` taking a **length**, while almost every other two-argument range in C++ takes a half-open `[begin, end)`, is a genuine inconsistency and a genuine bug source.

1.9 Complexity, all in one place

OPERATION	VECTOR	NOTE
<code>v[i]</code>	$O(1)$	
<code>push_back</code> / <code>pop_back</code>	$O(1)$ amortised	doubling
<code>insert</code> / <code>erase</code> at position <code>i</code>	$O(n - i)$	shifting
<code>insert</code> / <code>erase</code> at the front	$O(n)$	use <code>deque</code> if you need this often
<code>sort</code>	$O(n \log n)$	introsort, in place
<code>binary_search</code> / <code>lower_bound</code>	$O(\log n)$	sorted input only
<code>find</code> (linear)	$O(n)$	
<code>accumulate</code> , <code>max_element</code> , <code>count</code>	$O(n)$	
copy / assign	$O(n)$	

PART 2 — The index mental model

Linked lists were about pointers. Trees were about recursion. **Arrays are about indices**, and almost every array bug is an index bug. This part is the antidote. It is short, it is boring, and it will save you more marks than any algorithm in Part 3 onwards.

2.1 Every index must have a sentence

Before you write a loop, finish this sentence out loud: "**i** is the index of the first element I have not yet ...". If you cannot finish it, you are about to write a bug.

Compare two versions of "remove all copies of `val`":

```
// No sentence. Panic indexing.
int k = 0;
for (int i = 0; i < n; i++) if (v[i] != val) { v[k] = v[i]; k++; }
return k;
```

Same code, but now with the sentences attached — and it becomes impossible to get wrong:

```
// i = the index I am READING      -> "everything before i has been examined"
// k = the index I will WRITE next  -> "v[0..k-1] is the finished answer"
int k = 0;
for (int i = 0; i < n; i++) {
    if (v[i] != val) v[k++] = v[i];
}
return k;                          // k is both the next write slot AND the new length
```

That second pair of sentences is called a **loop invariant**: a statement that is true before the loop, stays true after every iteration, and therefore is true at the end — where it hands you the answer. `k` is the new length precisely *because* the invariant says `v[0..k-1]` is finished.

Write the invariant as a comment above the loop. In a live interview, say it aloud. It is the single clearest signal that you understand your own code, and it doubles as the correctness proof they are hoping you will give.

2.2 Half-open ranges: `[l, r)`

C++ ranges are **half-open**: the left end is included, the right end is not. `v.begin(), v.end()` is `[0, n)`. Adopt the same convention in your own logic and three good things follow:

- the **length is just** `r - l` — no `+1`, nowhere to put an off-by-one
- an **empty range is** `l == r`, a single natural termination test
- **splitting is seamless**: `[l, m)` and `[m, r)` cover `[l, r)` exactly once, with no element lost or doubled at the boundary

```
v = [ 10 , 20 , 30 , 40 , 50 ]
idx  0   1   2   3   4
[1,4) → 20, 30, 40      length 4-1 = 3
```

The two places you must **not** use half-open are binary search written as `[lo, hi]` inclusive (Part 6 explains exactly when each form is right) and two-pointer loops that walk toward each other, where `while (l < r)` already means "at least two elements left".

2.3 The four loop skeletons, and when each is right

```
for (int i = 0; i < n; i++)           // every element
for (int i = 1; i < n; i++)           // every element AND its predecessor: v[i-1], v[i]
for (int i = 0; i + 1 < n; i++)       // every ADJACENT PAIR: v[i], v[i+1]
for (int i = n - 1; i >= 0; i--)      // right to left
```

The third one is the important habit. **Write `i + 1 < n`, never `i < n - 1`**. They are equivalent for a non-empty array, but if `n` is an `unsigned` zero, `n - 1` explodes into a huge number and your loop runs off the end. `i + 1 < n` is safe for every `n` including 0.

Similarly, the guard goes **before** the access, and `&&` short-circuits so the order actually protects you:

```
if (i + 1 < n && v[i] == v[i + 1])    // safe: never evaluates v[i+1] out of bounds
if (v[i] == v[i + 1] && i + 1 < n)    // BUG: reads out of bounds, then checks
```

2.4 The five inputs you dry-run, every single time

Before you submit, run your code in your head on these. They are ordered by how often they break things:

#	INPUT	WHAT IT CATCHES
1	<code>[]</code> — empty	Missing <code>if (n == 0)</code> , <code>v[0]</code> on nothing, <code>n-1</code> underflow
2	<code>[5]</code> — one element	Loops that assume a pair exists, <code>l < r</code> that never runs
3	<code>[5,5,5,5]</code> — all equal	Duplicate handling, <code><</code> vs <code><=</code> , infinite two-pointer loops
4	<code>[1,2,3]</code> and <code>[3,2,1]</code> — sorted both ways	Direction, already-answered cases
5	The answer at index <code>0</code> and at index <code>n-1</code>	Off-by-one at both ends

A fifty-line solution that fails on `[]` scores the same as no solution. Thirty seconds of dry-running the empty case is the highest-return habit in competitive programming.

2.5 Overflow: the silent wrong answer

`int` holds roughly $\pm 2.1 \times 10^9$. Arrays make it very easy to exceed that without noticing:

```
int sum = 0;
for (int x : v) sum += x;           // n=10^5, x=10^5 -> 10^10. OVERFLOW.
```

The fixes, in order of preference:

```

long long sum = 0; // holds ~9.2 * 10^18. Just use it for any total.
accumulate(v.begin(), v.end(), 0LL);
int mid = lo + (hi - lo) / 2; // instead of (lo+hi)/2, which overflows near INT_MAX
long long prod = 1LL * a * b; // force 64-bit BEFORE the multiply, not after

```

`long long prod = a * b;` does not help: the multiplication happens in `int`, overflows, and the already-wrong value is then widened. Cast one operand first, with `1LL *` or `(long long)`.

Useful constants: `INT_MAX`, `INT_MIN`, `LLONG_MAX`, `LLONG_MIN` from `<climits>`. Do not initialise a running maximum to `0` unless the problem guarantees non-negative values — use `INT_MIN`, or better, initialise from `v[0]` and start the loop at `i = 1`.

2.6 In-place means O(1) extra space

"In-place" and "O(1) extra space" mean: **you may use a constant number of variables, no matter how big `n` is**. You may reuse the input array itself. You may not allocate a second array of length `n`.

Three techniques give you O(1) space over and over, and each has a part of this guide behind it:

1. **A write pointer** — overwrite the front of the array with the answer as you scan (Part 3).
2. **Swapping** — send each element where it belongs instead of copying it out (Part 3, Part 7).
3. **Encoding in the data itself** — the sign bit as a "seen" flag, or the first row of a matrix as a marker row (Part 7, Part 8).

The recursion stack counts as space. A recursive solution that goes `n` deep is O(n) space even though you allocated nothing, and that is why `reverse` by recursion is a wrong answer to an "O(1) space" question.

PART 3 — Two pointers

Two pointers is the pattern that turns $O(n^2)$ into $O(n)$. It is the most common single idea in placement array rounds, and it comes in exactly three shapes. Learn to tell them apart and you have covered a third of this topic.

SHAPE	POINTERS START	THEY MOVE	USE WHEN
A. Opposite ends	<code>l = 0, r = n-1</code>	toward each other	array is sorted , or you compare the two ends
B. Same direction (read/write)	<code>read = 0, write = 0</code>	both forward, <code>write</code> lags	filtering or compacting in place
C. Fast & slow	<code>slow = 0, fast = 1</code>	both forward, different speeds	duplicates, cycle detection, partitioning

3.1 Shape A — closing in from both ends

The controlling insight: **in a sorted array, the sum of the two ends tells you which end is wrong.**

```
// Two Sum on a SORTED array: return the 0-based indices, or {-1,-1}
vector<int> twoSumSorted(const vector<int>& v, int target) {
    int l = 0, r = (int)v.size() - 1;
    while (l < r) {
        int sum = v[l] + v[r];
        if (sum == target) return {l, r};
        if (sum < target) l++;           // need MORE -> only the left end can grow
        else r--;                       // need LESS -> only the right end can shrink
    }
    return {-1, -1};
}
```

Why is it correct to throw away a whole end? Because if `v[l] + v[r] < target`, then `v[l]` paired with *anything* at or below `r` is also too small — `v[r]` was the largest partner available. So `v[l]` can never be part of an answer, and discarding it discards no solution. **State that sentence in the interview.** That is the proof, and it is what separates "I memorised it" from "I understand it".

```
v = [2, 7, 11, 15], target = 18
  l      r      2+15=17 < 18 -> l++
    l      r      7+15=22 > 18 -> r--
      l      r      7+11=18 -> found (1,2)
```

Each step moves one pointer inward and neither ever goes back, so the loop runs at most `n` times: **$O(n)$ time, $O(1)$ space.**

`while (l < r)` not `l <= r`, because you need two *different* elements. If a problem allows reusing one element, then `l <= r`.

The family. Same skeleton, different body:

- **Reverse an array in place** — swap `v[l]`, `v[r]`, then `l++, r--`.

- **Palindrome check** — compare the ends instead of summing.
- **Container With Most Water** (LC 11) — area is `min(h[l],h[r]) * (r-l)`; move the *shorter* wall, because moving the taller one can only shrink the width while the height stays capped by the shorter.
- **Trapping Rain Water** (LC 42) — the same "move the shorter side" argument, keeping `leftMax` and `rightMax`.
- **3Sum** (LC 15) — sort, fix `i`, then run shape A on `[i+1, n)` for target `-v[i]`. $O(n^2)$.
- **Squares of a Sorted Array** (LC 977) — the largest square is at one of the two ends, so fill the output **from the back**.

3Sum is worth spelling out once, because the duplicate handling is where everyone loses the mark:

```
sort(v.begin(), v.end());
for (int i = 0; i + 2 < n; i++) {
    if (i > 0 && v[i] == v[i-1]) continue;           // skip duplicate FIRST elements
    int l = i + 1, r = n - 1;
    while (l < r) {
        long long s = (long long)v[i] + v[l] + v[r];
        if (s == 0) {
            res.push_back({v[i], v[l], v[r]});
            l++; r--;
            while (l < r && v[l] == v[l-1]) l++;      // skip duplicate SECOND elements
            while (l < r && v[r] == v[r+1]) r--;
        }
        else if (s < 0) l++;
        else r--;
    }
}
```

Three separate de-duplication points, and each compares against the element you just *left*, not the one you are about to visit. Sorting first is what makes duplicates adjacent and therefore skippable.

3.2 Shape B — the write pointer

This is the $O(1)$ -space workhorse. `write` is the index of the next slot in the answer; everything at `[0, write)` is final. `read` runs ahead over the original data. Because `write ≤ read` always, you never overwrite anything you have not already read.

```
// Remove every occurrence of val. Return the new length.
int removeElement(vector<int>& v, int val) {
    int write = 0;
    for (int read = 0; read < (int)v.size(); read++)
        if (v[read] != val) v[write++] = v[read];
    return write;
}
```

<code>v = [3, 2, 2, 3], val = 3</code>	
<code>read=0 v[0]=3 == val, skip</code>	<code>write=0 [_, 2, 2, 3]</code>
<code>read=1 v[1]=2 -> v[0]=2, write=1</code>	<code>write=1 [2, 2, 2, 3]</code>
<code>read=2 v[2]=2 -> v[1]=2, write=2</code>	<code>write=2 [2, 2, 2, 3]</code>
<code>read=3 v[3]=3 == val, skip</code>	
<code>return 2, and v[0..1] = [2,2]</code>	

Change one line and you have a different LeetCode problem:

CONDITION TO KEEP <code>v[read]</code>	PROBLEM
<code>v[read] != val</code>	LC 27 Remove Element
<code>write == 0 v[read] != v[write-1]</code>	LC 26 Remove Duplicates from Sorted Array
<code>write < 2 v[read] != v[write-2]</code>	LC 80 Remove Duplicates II (keep at most 2)
<code>v[read] != 0</code> (then zero-fill the tail)	LC 283 Move Zeroes

That third one is the beautiful generalisation: **"keep at most k copies"** is `write < k || v[read] != v[write-k]`. Compare against what you have already *accepted*, not against the input — that is what makes it work when the input has runs of length 9.

For Move Zeroes, note the two-pass version (compact, then fill the tail with zeros) and the one-pass version (`swap(v[write++], v[read])`) are both accepted; the swap version does more writes but keeps everything in one loop.

3.3 Shape C — fast & slow, and partitioning

Same direction, but now the pointers mean *regions* rather than read/write.

Dutch National Flag (LC 75 Sort Colors) is the canonical one, and it is the only three-pointer routine you must know by heart. Sort an array of 0s, 1s and 2s in one pass, in place:

```
void sortColors(vector<int>& v) {
    int low = 0, mid = 0, high = (int)v.size() - 1;
    // INVARIANT: [0,low) all 0s
    //             [low,mid) all 1s
    //             [mid,high] UNKNOWN
    //             (high,n) all 2s
    while (mid <= high) {
        if (v[mid] == 0) swap(v[low++], v[mid++]);
        else if (v[mid] == 1) mid++;
        else swap(v[mid], v[high--]); // note: mid does NOT move
    }
}
```

The one subtlety — and the only question they will ask about it — is **why `mid` advances after the 0-swap but not after the 2-swap**. When you swap with `low`, the value coming back is from the `[low, mid)` region, so it is a 1, already known-good: safe to move on. When you swap with `high`, the value coming back is from the unknown region and has never been looked at, so you must examine it on the next iteration. `while (mid <= high)`, inclusive, because `high` is still unknown territory.

Floyd's cycle detection on an array (LC 287) is the other fast/slow use. Treat `v[i]` as "the next index" and the array becomes a linked list; a duplicate value means two arrows point at the same node, which means a cycle. `slow = v[slow]`, `fast = v[v[fast]]`, then restart one pointer at 0 and walk both one step at a time; they meet at the entrance, which is the duplicate. $O(n)$ time, $O(1)$ space, input untouched.

3.4 Choosing the shape — the decision line

Sorted, or "pair/triplet summing to X"? → **Shape A**. Removing, filtering, or compacting in place? → **Shape B**. Partitioning into regions, or hunting a cycle? → **Shape C**.

And the prerequisite question, asked first every time: **am I allowed to sort?** Sorting costs $O(n \log n)$ and destroys the original indices. If the problem says "return the indices" (LC 1 Two Sum), sorting is usually wrong and the answer is a hash map instead — unless you sort `(value, index)` pairs, which is a valid but heavier alternative.

PART 4 — The sliding window

A window is a subarray `[l, r]` that you grow on the right and shrink on the left, never moving either pointer backwards. Because each index enters once and leaves once, the total work is **$O(n)$ even though the code has two nested-looking loops**. Say that sentence in the interview: "the inner `while` is amortised, each element is removed at most once."

The trigger words: *contiguous, subarray, substring, consecutive, window of size k, at most / at least K distinct, longest / shortest / maximum sum such that*

The disqualifier: if the array contains negative numbers and the condition is about a *sum*, the window does not work — growing the window no longer reliably increases the sum, so shrinking is no longer justified. That case is prefix sums plus a hash map (Part 5). This is the single most important boundary in the whole topic.

4.1 Fixed window of size k

Compute the first window, then roll it: add the entering element, subtract the leaving one.

```
// Maximum sum of any k consecutive elements.
int maxSumK(const vector<int>& v, int k) {
    int n = v.size();
    if (n < k) return 0;
    int sum = 0;
    for (int i = 0; i < k; i++) sum += v[i];
    int best = sum;
    for (int r = k; r < n; r++) {
        sum += v[r] - v[r - k];    // v[r] enters, v[r-k] leaves
        best = max(best, sum);
    }
    return best;
}
```

```
v = [2, 1, 5, 1, 3, 2], k = 3
[2 1 5] 1 3 2    sum = 8
2 [1 5 1] 3 2    sum = 8 - 2 + 1 = 7
2 1 [5 1 3] 2    sum = 7 - 1 + 3 = 9    <- best
2 1 5 [1 3 2]    sum = 9 - 5 + 2 = 6
```

`v[r - k]` is the element leaving: when `r` is the new right end, the window is `[r-k+1, r]`, so the one that just fell off is `r-k`. Derive it rather than memorise it; you will need the same reasoning for `k`-sized windows over strings and grids.

4.2 Variable window — the universal template

Almost every variable-window problem is this skeleton with two lines changed:

```

int l = 0;
/* window state, e.g. */ int sum = 0;
int best = 0;
for (int r = 0; r < n; r++) {
    /* 1. ADD v[r] to the window state */
    sum += v[r];

    while (/* 2. the window is INVALID */ sum > target) {
        /* 3. REMOVE v[l] from the window state */
        sum -= v[l];
        l++;
    }
    /* 4. here the window [l, r] is valid – record the answer */
    best = max(best, r - l + 1);
}
return best;

```

The four slots are always the same; only the *state* and the *invalid test* change. Note the window length is `r - l + 1` because both ends are inclusive here — this is the one place a closed range reads more naturally than a half-open one.

There are two flavours, and mixing them up is the classic error:

YOU WANT THE LONGEST VALID WINDOW	YOU WANT THE SHORTEST VALID WINDOW
<code>while (invalid) shrink;</code>	<code>while (valid) { record; shrink; }</code>
record the answer after the while	record the answer inside the while
shrinking restores validity	shrinking hunts for the minimum
LC 3, LC 424, LC 1004	LC 209, LC 76

```

// LC 209 – shortest subarray with sum >= target (positive numbers only)
int minSubArrayLen(int target, const vector<int>& v) {
    int n = v.size(), l = 0, sum = 0, best = INT_MAX;
    for (int r = 0; r < n; r++) {
        sum += v[r];
        while (sum >= target) { // VALID -> record, then try shorter
            best = min(best, r - l + 1);
            sum -= v[l++];
        }
    }
    return best == INT_MAX ? 0 : best;
}

```

4.3 Windows over characters — the frequency array

For strings, the window state is a count table. Use `int cnt[26]` or `int cnt[128]` rather than a hash map — it is a constant-size array, so it is faster and it never has an iterator-invalidation surprise.

```
// LC 3 – longest substring with no repeated character
int lengthOfLongestSubstring(const string& s) {
    int cnt[128] = {0};
    int l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        cnt[(unsigned char)s[r]]++;
        while (cnt[(unsigned char)s[r]] > 1) cnt[(unsigned char)s[l++]]--;
        best = max(best, r - l + 1);
    }
    return best;
}
```

The invalid test is specifically `cnt[s[r]] > 1` — only the character that just arrived can have created a duplicate, so you do not need to scan the whole table. That observation is what keeps it $O(n)$ instead of $O(26n)$.

For **LC 424 Longest Repeating Character Replacement**, the window is valid when `(window length) - (count of the most frequent character) <= k`. The famous trick is that you may keep `maxCount` as a running maximum and never decrease it; the window then never shrinks below the best answer found so far, and the result is still correct because a smaller `maxCount` could not produce a longer answer. If that argument does not sit comfortably yet, recompute the max over 26 entries — it is $O(26n)$, still fast enough, and correct for obvious reasons.

4.4 The "at most K" trick — counting subarrays

When a problem asks for the **number of subarrays with exactly K** of something, a window cannot answer it directly. Use the identity:

$$\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K-1)$$

and `atMost(K)` is a straightforward window, because the number of valid subarrays ending at `r` is exactly `r - l + 1`:

```
// LC 992-style: count subarrays with at most K distinct values
long long atMost(const vector<int>& v, int K) {
    unordered_map<int,int> cnt;
    long long res = 0;
    int l = 0;
    for (int r = 0; r < (int)v.size(); r++) {
        if (++cnt[v[r]] == 1) K--;
        while (K < 0) if (--cnt[v[l++]] == 0) K++;
        res += r - l + 1; // every subarray ending at r is valid
    }
    return res;
}
```

`res += r - l + 1` is the line to understand. With the window `[l, r]` valid and maximal, the valid subarrays that *end* at `r` are exactly those starting at `l, l+1, ..., r` — and there are `r - l + 1` of them. The same identity solves LC 930 (binary subarrays with sum K) and LC 1248 (nice subarrays), by mapping the condition onto a count.

4.5 Window checklist

Before you write a window, answer these four:

1. **What is the window state?** A sum, a count table, a distinct-count, a deque of indices.
2. **What makes it invalid?** One boolean over that state.
3. **Am I maximising or minimising?** Decides where the answer is recorded.
4. **Are there negative numbers and is the condition about a sum?** If yes — **abandon the window**, go to Part 5.

PART 5 — Prefix sums and difference arrays

A prefix sum answers "what is the sum of `v[i..j]`?" in $O(1)$ after $O(n)$ of preparation. It is the tool for every range-sum question, and combined with a hash map it solves the whole family of subarray-sum problems that sliding windows cannot touch.

5.1 The construction, and the one formula

Define `P[i]` = sum of the first `i` elements, so `P[0] = 0` and `P[i] = v[0] + ... + v[i-1]`.

```
int n = v.size();
vector<long long> P(n + 1, 0);
for (int i = 0; i < n; i++) P[i + 1] = P[i] + v[i];

// sum of v[i..j] inclusive:
long long s = P[j + 1] - P[i];
```

```
v =      [ 3 , 1 , 4 , 1 , 5 ]
P = [ 0 , 3 , 4 , 8 , 9 , 14 ]
      P0  P1  P2  P3  P4  P5
```

sum of `v[1..3]` = `1+4+1` = `6` = `P[4] - P[1]` = `9 - 3`

Make `P` have length `n+1` with `P[0] = 0`. The extra leading zero is not decoration — it is what lets a range starting at index 0 be written with the same formula as every other range, with no special case. Every time you see someone struggling with `if (i == 0) ... else ...` in a prefix-sum solution, they built `P` with length `n`.

Use `long long` for `P`. The whole point is summing many things.

5.2 Prefix + hash map — the pattern windows cannot do

Here is the reformulation that unlocks a dozen problems:

a subarray `v[i..j]` sums to `k` $\iff$ `P[j+1] - P[i] = k` $\iff$ `P[i] = P[j+1] - k`

So scan left to right keeping every prefix value you have seen in a hash map. At each position, the number of subarrays ending here with sum `k` is the number of times `current - k` has appeared before.

```
// LC 560 – count subarrays summing to k. Works with NEGATIVE numbers.
int subarraySum(const vector<int>& v, int k) {
    unordered_map<long long, int> seen;
    seen[0] = 1; // the empty prefix – do not forget this
    long long pre = 0;
    int count = 0;
    for (int x : v) {
        pre += x;
        auto it = seen.find(pre - k);
        if (it != seen.end()) count += it->second;
        seen[pre]++;
    }
    return count;
}
```

seen[0] = 1 is the line everyone forgets. Without it, any subarray that starts at index 0 is missed, because its required earlier prefix is the empty one. Test with `v = [3], k = 3`: the answer must be 1, and without that line you get 0.

Note the order: **look up before you insert**. Inserting first would let a subarray of length zero match when `k = 0`.

The variants all reuse the same skeleton with a different quantity accumulated in `pre`:

PROBLEM	PRE ACCUMULATES	MAP KEY
LC 560 subarray sum = k	running sum	<code>pre - k</code>
LC 974 sum divisible by k	<code>((pre % k) + k) % k</code>	the same remainder
LC 525 equal 0s and 1s	+1 for 1, -1 for 0	the same value (first index)
LC 1 Two Sum	the element itself	<code>target - x</code>
XOR subarrays	running XOR	<code>pre ^ k</code>

The 0/1 → -1/+1 substitution in LC 525 is worth internalising: **"equal counts of A and B" becomes "sum is zero"**, and "sum is zero" is a prefix-map problem. That trick recurs constantly.

For longest-rather-than-count problems, store the **first** index at which each prefix appeared (`if (!mp.count(pre)) mp[pre] = i;`) — the earliest start gives the longest subarray.

5.3 Prefix products, and the no-division rule

LC 238 Product of Array Except Self asks for `res[i] = product of everything but v[i]`, with no division allowed (because of zeros) and O(1) extra space beyond the output.

```
vector<int> productExceptSelf(const vector<int>& v) {
    int n = v.size();
    vector<int> res(n, 1);
    for (int i = 1; i < n; i++) res[i] = res[i-1] * v[i-1]; // prefix products
    int suf = 1;
    for (int i = n - 1; i >= 0; i--) { res[i] *= suf; suf *= v[i]; }
    return res;
}
```

Two sweeps: the first leaves "product of everything to my left" in `res`, the second multiplies in "product of everything to my right", carried in a single variable. **The left-then-right double sweep is a pattern in its own right** — the same shape solves Trapping Rain Water with arrays, Candy, and every "combine information from both sides" question.

5.4 The difference array — range updates in $O(1)$

The mirror image of a prefix sum. When you must apply many "add `x` to every element in `[l, r]`" updates and only read the array at the end, do not touch the range: record the change at its two boundaries.

```
vector<long long> diff(n + 1, 0);
// add x to v[l..r]:
diff[l] += x;
diff[r + 1] -= x;           // r+1 is why diff has length n+1
// ...after ALL updates, one prefix sum recovers the array:
long long run = 0;
for (int i = 0; i < n; i++) { run += diff[i]; v[i] += run; }
```

`q` updates cost $O(q)$ instead of $O(q \cdot n)$, and one final $O(n)$ pass materialises the result. This is the trick behind **LC 1109 Corporate Flight Bookings**, **LC 370 Range Addition**, and most "meeting rooms / how many at once" counting questions. If the coordinates are large but sparse, the same idea with a sorted map of `(position, delta)` is the **sweep line**.

5.5 2D prefix sums

Same idea in a grid. `P[i][j]` = sum of the rectangle from `(0,0)` to `(i-1, j-1)`, with a zero row and zero column at the top and left.

```
vector<vector<long long>> P(r + 1, vector<long long>(c + 1, 0));
for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
        P[i+1][j+1] = g[i][j] + P[i][j+1] + P[i+1][j] - P[i][j];

// sum of the rectangle (r1,c1)..(r2,c2) inclusive:
long long s = P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1];
```

A	B
C	D

Both subtractions remove the top-left corner A, so it has been taken away twice and must be added back once.
 $D = \text{Total} - B' - C' + A$
 (inclusion-exclusion)

That `+ P[r1][c1]` is inclusion-exclusion, and it is the entire content of the question when an interviewer asks about 2D prefix sums. Solves LC 304 (range sum query 2D) and the matrix versions of "max sum submatrix".

PART 6 — Binary search on arrays

Binary search is where sorted arrays pay off. Everyone has written it; almost no one has written it without an off-by-one. This part gives you **two templates** — memorise them exactly, and never improvise the boundaries again.

6.1 Template 1 — find an exact value

Closed range `[lo, hi]`. Both ends are candidates, so the loop must run while `lo <= hi`.

```
int binarySearch(const vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo <= hi) {                // <= : lo==hi is still a candidate
        int mid = lo + (hi - lo) / 2; // never (lo+hi)/2 - overflow
        if (v[mid] == target) return mid;
        if (v[mid] < target) lo = mid + 1; // +1 and -1 guarantee progress
        else hi = mid - 1;
    }
    return -1;
}
```

The three rules that make it terminate and be correct:

1. `while (lo <= hi)` pairs with `mid ± 1`. `while (lo < hi)` pairs with a `mid / mid+1` split. **Never mix them** — mixing is what produces infinite loops.
2. `mid = lo + (hi - lo) / 2` avoids `lo + hi` overflowing `int`. This exact bug sat in the JDK for nine years.
3. Integer division rounds **down**, so `mid` is the left of the two middles when the range is even. That matters in template 2.

6.2 Template 2 — find a boundary (the one that actually matters)

Most real problems are not "find 7". They are "find the **first** position where something becomes true" — the first element $\geq$ target, the first bad version, the smallest workable capacity. Use a half-open range `[lo, hi)` and converge to a single index:

```
// first index i in [0,n] with pred(i) true, where pred is false...false,true...true
int lowerBoundIdx(const vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size(); // hi = n, NOT n-1: "not found" is a valid answer
    while (lo < hi) {                // < : stop when the range is a single point
        int mid = lo + (hi - lo) / 2;
        if (v[mid] >= target) hi = mid; // mid might be the answer - KEEP it
        else lo = mid + 1;              // mid is definitely not - discard it
    }
    return lo;                       // lo == hi == the first true position
}
```

Read it as a **partition**: the predicate is false on a prefix and true on a suffix, and you are hunting the seam. `hi = mid` (not `mid - 1`) because `mid` is still a candidate. The loop terminates because when `hi - lo == 1`, `mid == lo`, and each branch strictly shrinks the range.

Once you have it, the standard queries fall out:

WANT	PREDICATE	STL EQUIVALENT
First index with <code>v[i] >= x</code>	<code>v[mid] >= x</code>	<code>lower_bound(b, e, x) - b</code>
First index with <code>v[i] > x</code>	<code>v[mid] > x</code>	<code>upper_bound(b, e, x) - b</code>
Number of elements equal to <code>x</code>	—	<code>upper_bound - lower_bound</code>
Last index with <code>v[i] <= x</code>	—	<code>upper_bound(b,e,x) - b - 1</code>
Insert position keeping sorted	<code>v[mid] >= x</code>	<code>lower_bound</code>

In a contest, just call `lower_bound` / `upper_bound`. In an interview, be ready to write template 2 on the board — that is exactly what they are checking. And remember `lower_bound` can return `v.end()`; always test `it != v.end()` before dereferencing.

6.3 Rotated sorted arrays

A sorted array rotated at an unknown pivot is still searchable in $O(\log n)$, because **at least one half of any split is still properly sorted**. Find which, then decide whether the target lives in it.

```
// LC 33 – search in a rotated sorted array, distinct values
int search(const vector<int>& v, int target) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (v[mid] == target) return mid;
        if (v[lo] <= v[mid]) { // LEFT half is sorted
            if (v[lo] <= target && target < v[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // RIGHT half is sorted
            if (v[mid] < target && target <= v[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

```
[4, 5, 6, 7, 0, 1, 2]    target = 0
 lo      mid      hi    v[lo]=4 <= v[mid]=7 -> left half sorted
                        0 not in [4,7) -> go right, lo = mid+1
    lo mid hi    v[lo]=0 <= v[mid]=1 -> left sorted, 0 in [0,1) -> hi = mid-1
                        found at index 4
```

`v[lo] <= v[mid]` with `<=` because when the range is two elements, `mid == lo` and the left half (one element) is trivially sorted. Two related problems:

- **LC 153 Find Minimum in Rotated Sorted Array** — no target at all. Compare `v[mid]` with `v[hi]`: if `v[mid] > v[hi]` the minimum is strictly right (`lo = mid+1`), else it is at `mid` or left (`hi = mid`). Use the `lo < hi` form and return `v[lo]`. **Compare against hi, never lo** — comparing with `lo` fails on an unrotated array.
- **With duplicates** (LC 81 / 154) the guarantee breaks: `[3,3,3,1,3]` gives `v[lo] == v[mid] == v[hi]` and tells you nothing. The fix is `hi--` in that case, which makes the worst case $O(n)$. Be ready to say that out loud — "duplicates degrade it to linear" is the expected answer.

6.4 Binary search on the answer

The most powerful version, and the one that separates strong candidates. You are not searching the array at all — you are searching the **space of possible answers**, using a feasibility check.

It applies whenever the answer is a number in a known range and feasibility is **monotonic**: if `x` works then everything above it works (or everything below).

```
candidate answer: 1 2 3 4 5 6 7 8 9 10
feasible?         N N N N Y Y Y Y Y Y
                  ↑ binary search for this seam
```

```
// LC 875 - Koko eating bananas: smallest speed that finishes within h hours
bool canFinish(const vector<int>& piles, long long speed, int h) {
    long long hours = 0;
    for (int p : piles) hours += (p + speed - 1) / speed; // ceil division
    return hours <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
    long long lo = 1, hi = *max_element(piles.begin(), piles.end());
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (canFinish(piles, mid, h)) hi = mid; // feasible -> try slower
        else lo = mid + 1;
    }
    return (int)lo;
}
```

The recipe, every time:

1. **Identify the answer range** `[lo, hi]` — usually `max(v)` to `sum(v)`, or 1 to `max(v)`.
2. **Write** `bool feasible(x)` as a simple $O(n)$ greedy loop. This is where the real thinking is.
3. **Argue monotonicity** in one sentence. Without it the search is invalid.
4. Run template 2. Total cost **$O(n \log(\text{range}))$** .

The family: LC 875 Koko, LC 1011 Capacity to Ship Packages, LC 410 Split Array Largest Sum, LC 1482 Bouquets, LC 719/378 kth smallest distance or matrix element. They are all the same code with a different `feasible`.

The tell: "minimise the maximum ...", "maximise the minimum ...", "smallest capacity such that ...". Whenever you see one of those phrasings, stop looking for a clever formula and go straight to binary search on the answer.

`(p + speed - 1) / speed` is ceiling division for positive integers. Memorise that idiom — you will use it in a third of these problems.

6.5 LC 4 — median of two sorted arrays

The hardest classic in this family, and it is binary search on a **partition** rather than on a value. Choose how many elements of the answer's left half come from the shorter array; that fixes the count from the other array; then check the four boundary values `L1, R1, L2, R2`. The partition is correct when `L1 <= R2 && L2 <= R1`.

Always binary search the **shorter** array so the derived count cannot go out of range, and use `INT_MIN / INT_MAX` as the boundary sentinels when a partition is empty at one end. $O(\log(\min(m,n)))$. The full code is in the companion solved-questions book.

PART 7 — Hashing, frequency arrays, and using the array as its own hash table

This part is about trading memory for time — and then, in the last three sections, about getting the time without spending the memory. Those $O(1)$ -space tricks are the ones interviewers love, because there is no way to fake having seen them.

7.1 The frequency array — the fastest hash table there is

When the keys are small and dense (letters, digits, values `1..n`), do not reach for a hash map. Use a plain array indexed by the key. It is faster, it is cache-friendly, and it cannot rehash under you.

```
int cnt[26] = {0}; // lowercase letters
for (char ch : s) cnt[ch - 'a']++;

int cnt[128] = {0}; // any ASCII character
for (char ch : s) cnt[(unsigned char)ch]++;

vector<int> cnt(n + 1, 0); // values known to be in 1..n
for (int x : v) cnt[x]++;
```

`ch - 'a'` works because the letters are consecutive in ASCII. Cast to `unsigned char` before indexing with a raw character: `char` is signed on most compilers, so a byte above 127 becomes negative and you index out of bounds.

Anagram check, in six lines and $O(n)$:

```
bool isAnagram(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    int cnt[26] = {0};
    for (char ch : a) cnt[ch - 'a']++;
    for (char ch : b) if (--cnt[ch - 'a'] < 0) return false;
    return true;
}
```

Increment on one pass, decrement on the other, and bail the moment a count goes negative — no second loop needed to verify.

7.2 `unordered_map` and `unordered_set`

For sparse or large keys.

```
unordered_map<int, int> mp;
mp[x]++; // inserts 0 then increments – no need to check first
if (mp.count(x)) ... // membership
auto it = mp.find(x);
if (it != mp.end()) use(it->second); // one lookup instead of two
for (auto& [key, val] : mp) ... // C++17 structured bindings
unordered_set<int> seen;
if (!seen.insert(x).second) { /* x was already there */ }
```

	UNORDERED_MAP	MAP
Average lookup	O(1)	O(log n)
Worst case	O(n) (collisions)	O(log n)
Order	none	sorted by key
Use when	you just need membership/counts	you need order, floor/ceil, or ranges

Two things worth knowing:

- `mp[x]` **inserts**. Merely *reading* `mp[x]` on a missing key creates it with value 0 and grows the map. When you only want to test, use `count` or `find`. In a loop over a `const` map, `mp[x]` will not even compile — which is a feature.
- On Codeforces, `unordered_map<int,int>` can be attacked with anti-hash tests and degrade to O(n) per operation. For placements it is fine; if a solution mysteriously times out, swapping to `map` or adding a custom hash is the known fix.

7.3 Index-as-hash — when the values are `1..n`

Here is the observation that powers a whole family of interview favourites:

If an array of length `n` contains values in `1..n` (or `0..n-1`), then **the value itself is a valid index**. The array can be its own hash table, and you need no extra memory at all.

Three ways to use it:

(a) Cyclic sort — put each value where it belongs. `v[i]` should hold `i + 1`. Keep swapping the element at `i` to its home until the one sitting at `i` is already correct.

```
// LC 41 — first missing positive, O(n) time, O(1) space
int firstMissingPositive(vector<int>& v) {
    int n = v.size();
    for (int i = 0; i < n; i++)
        while (v[i] > 0 && v[i] <= n && v[v[i] - 1] != v[i])
            swap(v[i], v[v[i] - 1]);
    for (int i = 0; i < n; i++) if (v[i] != i + 1) return i + 1;
    return n + 1;
}
```

The `while` looks like it could make this O(n²), but each successful swap puts one value permanently in its correct home, so there are at most `n` swaps across the entire outer loop: **O(n) total**. Have that argument ready — it is the follow-up question, guaranteed.

The loop condition `v[v[i]-1] != v[i]` compares *values*, not indices. Writing `v[v[i]-1] != i+1` instead gives an **infinite loop** when duplicates are present, because two equal values keep swapping with each other forever.

(b) Negation marking — use the sign bit as a "seen" flag. When all values are positive and in `1..n`, mark "I have seen value `x`" by making `v[x-1]` negative. Read with `abs()`.

```
// LC 448 — all numbers in 1..n missing from the array, O(1) space
vector<int> findDisappeared(vector<int>& v) {
    int n = v.size();
    for (int i = 0; i < n; i++) {
        int idx = abs(v[i]) - 1;          // abs - v[i] may already be marked
        if (v[idx] > 0) v[idx] = -v[idx];
    }
    vector<int> res;
    for (int i = 0; i < n; i++) if (v[i] > 0) res.push_back(i + 1);
    return res;
}
```

`abs(v[i])` is the whole trick, and forgetting it is the bug. By the time you reach index `i`, `v[i]` may itself have been flipped negative by an earlier mark, and `-3 - 1 = -4` is a fine way to corrupt memory. The same code with `if (v[idx] < 0) res.push_back(abs(v[i]))` solves **LC 442 Find All Duplicates**.

If the problem says the input must not be modified, this technique is off the table — say so, and offer the $O(n)$ extra-space version instead.

(c) Floyd's cycle detection — covered in Part 3.3. LC 287, when the array is read-only and $O(1)$ space is demanded.

7.4 XOR — the parity trick

Three properties, and every XOR puzzle is a consequence:

```
x ^ x = 0      a value cancels itself
x ^ 0 = x      zero is the identity
XOR is commutative and associative -> order does not matter
```

```
// LC 136 — every element appears twice except one. Find it. O(1) space.
int singleNumber(const vector<int>& v) {
    int res = 0;
    for (int x : v) res ^= x;          // pairs cancel; the loner survives
    return res;
}

// LC 268 — missing number from 0..n
int missingNumber(const vector<int>& v) {
    int res = v.size();
    for (int i = 0; i < (int)v.size(); i++) res ^= i ^ v[i];
    return res;
}
```

For LC 268 the sum formula `n*(n+1)/2 - sum` also works and is easier to explain — but it can overflow for large `n`, while XOR never can. Offer both; mention the overflow difference. That comparison is exactly the kind of remark that lands well in an interview.

LC 260 (two numbers appear once) extends it: XOR everything to get `a^b`, isolate any set bit with `d = x & -x`, and partition the array by that bit so each group contains exactly one of the two. Isolating the lowest set bit is worth memorising on its own.

7.5 Sorting as a preprocessing step

Sorting costs $O(n \log n)$ and buys you a lot:

- duplicates become **adjacent** — one linear scan finds them
- two pointers become possible
- binary search becomes possible
- "group the equal ones" becomes trivial
- intervals become sweepable (Part 8)

Do not sort when: you need the original indices (unless you sort pairs), the input is already $O(n)$ -solvable with a hash map, or the problem demands $O(n)$ and `n` is large. **Sorting to find duplicates is $O(n \log n)$; a hash set does it in $O(n)$.** Knowing when *not* to sort is as examinable as knowing when to.

Two canonical uses:

```
// LC 128 - longest consecutive sequence, O(n) WITHOUT sorting
int longestConsecutive(const vector<int>& v) {
    unordered_set<int> s(v.begin(), v.end());
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;           // only start counting at a sequence START
        int len = 1;
        while (s.count(x + len)) len++;
        best = max(best, len);
    }
    return best;
}
```

The `if (s.count(x-1)) continue;` line is what keeps this $O(n)$ rather than $O(n^2)$: the inner `while` only ever runs from the smallest element of a run, so across the whole loop each element is visited at most once by an inner walk. If an interviewer challenges the complexity, that is the sentence to give.

PART 8 — Kadane, intervals, and matrix manipulation

Three self-contained families that appear in almost every placement paper.

8.1 Kadane's algorithm

LC 53 Maximum Subarray: the largest sum of any contiguous subarray. The whole algorithm is one decision, made at every index:

At index `i`, either **extend** the best subarray ending at `i-1`, or **start fresh** at `i`. Take whichever is larger.

```
int maxSubArray(const vector<int>& v) {
    int best = v[0], cur = v[0];           // start from v[0], NOT from 0
    for (int i = 1; i < (int)v.size(); i++) {
        cur = max(v[i], cur + v[i]);       // extend, or restart here
        best = max(best, cur);
    }
    return best;
}
```

```
v =    [-2,  1, -3,  4, -1,  2,  1, -5,  4]
cur =  [-2,  1, -2,  4,  3,  5,  6,  1,  5]
best =  [-2,  1,  1,  4,  4,  5,  6,  6,  6]    <- answer 6, from [4,-1,2,1]
```

Initialise from `v[0]`, never from `0`. With `best = 0` an all-negative array returns 0, which is not a subarray sum unless the empty subarray is allowed. This is the single most common Kadane bug and it is exactly what the hidden test `[-3,-1,-2]` checks — the answer is `-1`.

To also report *where* the subarray is: remember `start` when `cur` restarts, and commit `(start, i)` whenever `best` improves.

LC 152 Maximum Product Subarray is the same shape with one extra idea. A large negative times a negative becomes a large positive, so you must carry **both** the running maximum and the running minimum, and swap them when the current element is negative:

```
int maxProduct(const vector<int>& v) {
    int best = v[0], hi = v[0], lo = v[0];
    for (int i = 1; i < (int)v.size(); i++) {
        if (v[i] < 0) swap(hi, lo);       // negative flips the roles
        hi = max(v[i], hi * v[i]);
        lo = min(v[i], lo * v[i]);
        best = max(best, hi);
    }
    return best;
}
```

Zeros take care of themselves: `hi` and `lo` both collapse to 0 and the next element restarts the run.

LC 121 Best Time to Buy and Sell Stock is Kadane in disguise — track the minimum price seen so far and the best profit against it. One pass, $O(1)$ space. If you write it as "Kadane over the differences"

you get the same answer, and saying that out loud shows you see the connection.

8.2 Intervals

An interval is a pair `[start, end]`. Nearly every interval problem starts with the same line:

```
sort(iv.begin(), iv.end());    // sorts by start, then by end – usually what you want
```

LC 56 Merge Intervals: sort by start, then sweep, extending the last kept interval whenever the next one overlaps it.

```
vector<vector<int>> merge(vector<vector<int>>& iv) {
    sort(iv.begin(), iv.end());
    vector<vector<int>> res;
    for (auto& cur : iv) {
        if (!res.empty() && cur[0] <= res.back()[1])
            res.back()[1] = max(res.back()[1], cur[1]);    // max – cur may be CONTAINED
        else
            res.push_back(cur);
    }
    return res;
}
```

Two details carry the marks. `cur[0] <= res.back()[1]` uses `<=` because touching intervals like `[1,4]` and `[4,5]` are normally considered overlapping — check the statement, because some problems say otherwise. And `max(res.back()[1], cur[1])` rather than plain assignment, because `[1,10]` followed by `[2,3]` must stay `[1,10]`.

The rest of the family:

PROBLEM	SORT BY	THEN
LC 56 Merge Intervals	start	extend or push
LC 57 Insert Interval	already sorted	three phases: before, merge, after
LC 435 Non-overlapping Intervals	end	greedily keep the earliest finisher
LC 252/253 Meeting Rooms	start	min-heap of end times, or a +1/−1 sweep

The rule of thumb: maximise how many intervals you keep → sort by **end**. Merge or combine them → sort by **start**. Sorting by end for the greedy is the classic activity-selection argument: finishing earliest leaves the most room for everything after it.

For LC 253 (how many rooms are needed at once), the difference-array sweep from Part 5.4 is the cleanest solution: +1 at each start, −1 at each end, prefix-sum and take the maximum.

8.3 Matrix manipulation

Three set pieces. All three are pure index discipline — no algorithm, just care.

LC 48 Rotate Image — 90° clockwise, in place. Transpose, then reverse each row.

```

void rotate(vector<vector<int>>& g) {
    int n = g.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) // j = i+1 : upper triangle ONLY
            swap(g[i][j], g[j][i]);
    for (auto& row : g) reverse(row.begin(), row.end());
}

```

`j = i + 1` is the whole trick. Starting `j` at 0 swaps every pair twice and leaves the matrix exactly as it was. For anticlockwise: transpose, then reverse the **columns** (i.e. reverse the order of the rows).

LC 54 Spiral Matrix. Keep four shrinking boundaries and peel one ring at a time.

```

vector<int> spiralOrder(vector<vector<int>>& g) {
    vector<int> res;
    if (g.empty()) return res;
    int top = 0, bot = g.size() - 1, left = 0, right = g[0].size() - 1;
    while (top <= bot && left <= right) {
        for (int j = left; j <= right; j++) res.push_back(g[top][j]);
        top++;
        for (int i = top; i <= bot; i++) res.push_back(g[i][right]);
        right--;
        if (top <= bot) { // guard: single row left
            for (int j = right; j >= left; j--) res.push_back(g[bot][j]);
            bot--;
        }
        if (left <= right) { // guard: single column left
            for (int i = bot; i >= top; i--) res.push_back(g[i][left]);
            left++;
        }
    }
    return res;
}

```

The two `if` guards are the entire difficulty. Without them, a matrix with one row left re-traverses it backwards and you emit duplicates. Test on `[[1,2,3]]` and `[[1],[2],[3]]` — those two inputs catch every buggy version.

LC 73 Set Matrix Zeroes, O(1) space. The naive fix (zero the row and column as soon as you see a zero) is wrong, because the zeros you write are indistinguishable from the zeros that were there. Buffering the rows and columns in two `vector<bool>` s is correct and O(m+n). To get O(1), **use the first row and first column as those buffers** — plus one extra flag, because `g[0][0]` would otherwise have to serve two masters:

```

bool firstColZero = false;
for (int i = 0; i < m; i++) {
    if (g[i][0] == 0) firstColZero = true;
    for (int j = 1; j < n; j++)
        if (g[i][j] == 0) { g[i][0] = 0; g[0][j] = 0; }
}
for (int i = m - 1; i >= 0; i--) { // BACKWARDS
    for (int j = n - 1; j >= 1; j--)
        if (g[i][0] == 0 || g[0][j] == 0) g[i][j] = 0;
    if (firstColZero) g[i][0] = 0;
}

```

The second pass runs **backwards** so the marker row and column are consumed last. Go forwards and you destroy the markers before you have used them.

8.4 The rotation trick — three reversals

LC 189 Rotate Array by `k` to the right, in $O(1)$ space:

```
void rotate(vector<int>& v, int k) {
    int n = v.size();
    k %= n;                                // k can exceed n
    reverse(v.begin(), v.end());
    reverse(v.begin(), v.begin() + k);
    reverse(v.begin() + k, v.end());
}
```

```
[1,2,3,4,5,6,7], k = 3
reverse all      -> [7,6,5,4,3,2,1]
reverse first 3  -> [5,6,7,4,3,2,1]
reverse the rest -> [5,6,7,1,2,3,4] ✓
```

`k %= n` first — `k` is often larger than `n` in the tests, and `v.begin() + k` past the end is undefined behaviour. The same triple reversal rotates a string, and is the standard answer to "rotate in place with $O(1)$ memory".

8.5 Boyer-Moore majority vote

LC 169: an element appears more than $n/2$ times; find it in $O(n)$ time and $O(1)$ space.

```
int majorityElement(const vector<int>& v) {
    int cand = v[0], count = 0;
    for (int x : v) {
        if (count == 0) cand = x;
        count += (x == cand) ? 1 : -1;
    }
    return cand;
}
```

The intuition to state: pair off each majority element with a different element and cancel them; since the majority has more than half the elements, something of it must survive. **The algorithm only works if a majority is guaranteed** — if it is not, run a second pass to verify the candidate actually occurs more than $n/2$ times. Interviewers ask for exactly that follow-up.

The $n/3$ variant (LC 229) keeps two candidates and two counters, and always needs the verification pass.

PART 9 — The complete pattern taxonomy

Fourteen patterns. Every array problem in a placement test is one of these, or two of them stacked.

#	PATTERN	RECOGNISE IT BY	COST	CANONICAL PROBLEM
1	Linear scan with state	"max/min/count as you go"	$O(n)$	LC 121 Best Time to Buy/Sell
2	Two pointers — opposite ends	sorted + pair/triplet, or compare both ends	$O(n)$	LC 167, LC 11, LC 15
3	Two pointers — write index	"in place", "return the new length"	$O(n)$	LC 26, LC 27, LC 283
4	Partitioning (Dutch flag)	rearrange into 2 or 3 groups, one pass	$O(n)$	LC 75 Sort Colors
5	Fixed sliding window	"window of size k", "every k consecutive"	$O(n)$	Max sum of k elements
6	Variable sliding window	longest/shortest contiguous satisfying X, non-negative	$O(n)$	LC 3, LC 209, LC 424
7	Prefix sum	many range-sum queries, or "sum of $v[i..j]$ "	$O(n)$ build, $O(1)$ query	LC 303, LC 304
8	Prefix sum + hash map	"count/longest subarray with sum k", negatives allowed	$O(n)$	LC 560, LC 525, LC 974
9	Difference array / sweep	many range updates, or "how many overlap at once"	$O(n + q)$	LC 1109, LC 253
10	Binary search on a sorted array	sorted input, "find / first / last / insert position"	$O(\log n)$	LC 34, LC 35, LC 33
11	Binary search on the answer	"minimise the maximum", "smallest capacity such that"	$O(n \log R)$	LC 875, LC 410, LC 1011
12	Hashing / frequency counting	membership, duplicates, counts, anagrams, "return indices"	$O(n)$	LC 1, LC 217, LC 128
13	Index-as-hash / in-place marking	values are <code>1..n</code> and $O(1)$ space demanded	$O(n)$	LC 41, LC 448, LC 287
14	Matrix / grid index manipulation	rotate, spiral, transpose, zeroing	$O(m \cdot n)$	LC 48, LC 54, LC 73

Two more that live in other guides but arrive in array clothing: the **monotonic stack** (next-greater, histogram — see the Stacks & Queues guide, Part 4) and the **monotonic deque** (sliding window maximum). If a problem asks "for each element, the next one bigger than it", you are in that guide, not this one.

Telling patterns 6 and 8 apart — the decision that matters most

They look identical in the statement: "*find the subarray with sum ...*". The distinguisher is **one line of the constraints**:

	VARIABLE WINDOW (6)	PREFIX + HASH (8)
Values	all non-negative	any , negatives included
Why it works	growing the window only increases the sum, so shrinking is justified	no monotonicity to exploit; look up the complement instead
Answers	longest / shortest	count, or longest
Space	O(1)	O(n)

Check for negative numbers before you choose. LC 209 (positive only) is a window; LC 560 (negatives allowed) cannot be. Candidates who write a window for LC 560 fail on `[1, -1, 0], k=0` and usually cannot see why.

Telling patterns 3 and 4 apart

Both rearrange in place. Pattern 3 has one output region and discards the rest — "keep the good ones". Pattern 4 has **two or three output regions** and everything is kept, just relocated. If nothing is thrown away, you want partitioning.

PART 10 — How to approach ANY array problem

Six steps. Follow them out loud in an interview — the narration is half the score.

Step 1 — Read the constraints before the statement

`n ≤ ?` tells you the target complexity (the table in Part 0.6). Are the values bounded? Can they be negative? Can the array be empty? Are there duplicates? Is it already sorted? **Is it sorted** is the single highest-value question, because it unlocks two pointers and binary search for free.

Step 2 — State the brute force, with its complexity

"Check all pairs — $O(n^2)$." Say it, do not code it. It anchors the conversation, proves you have understood the problem, and gives you a target to beat. Then ask the question that finds the pattern: **"what work am I repeating?"** Re-summing an overlapping range → prefix sums or a window. Re-searching for a complement → a hash map. Re-scanning a sorted region → two pointers or binary search.

Step 3 — Match a pattern from the table

Run the trigger words: *contiguous* → window or prefix. *Sorted* → two pointers or binary search. *In place / $O(1)$ space* → write index, swapping, or in-place marking. *Return indices* → hash map, not sorting. *Minimise the maximum* → binary search on the answer. *Values in $1..n$* → index-as-hash.

Step 4 — Name every index before you write the loop

For each pointer, finish the sentence from Part 2.1: "`l` is the start of the current window", "`write` is the next output slot", "`P[i]` is the sum of the first `i` elements". Write them as a comment. If you cannot finish a sentence, your pattern choice is wrong — go back to step 3 rather than pushing on.

Step 5 — Write the guards first, then the body

```
int n = v.size();
if (n == 0) return /* the right empty answer */;
```

Then `i + 1 < n` rather than `i < n - 1`, `long long` for any running total, `lo + (hi-lo)/2` for any midpoint, and bounds checked **before** the access with `&&`.

Step 6 — Dry-run the five inputs

`[]`, `[x]`, all-equal, sorted both ways, and the answer at each end. Trace the actual variable values on paper for at least the first two. This step finds more bugs than the previous five steps combined, and doing it visibly in an interview is a strong signal.

If you are stuck after four minutes: **sort it, or hash it**. Say "let me consider what sorting buys me" — sorting turns an unordered problem into a two-pointer one and often exposes the intended solution. Even when it is not the optimal answer, an $O(n \log n)$ solution you can explain beats an $O(n)$ one you cannot finish.

PART 11 — The bug catalogue

Fourteen bugs. Between them they account for nearly every wrong answer you will submit in this topic. When a solution fails, read this list before you reread your code.

- 1. Out-of-bounds by one.** `for (int i = 0; i <= n; i++)` or reading `v[i+1]` without a guard. *Fix:* `i < n`, and guard with `i + 1 < n &&` **before** the access.
 - 2. `i < n - 1` when `n` is unsigned.** `v.size() - 1` on an empty vector is 18446744073709551615, and the loop runs off the end. *Fix:* `int n = v.size();` on line one, and prefer `i + 1 < n`.
 - 3. Not handling the empty array.** `v[0]`, `v.back()`, `*max_element(...)`, or `k %= n` all break on `n == 0`. *Fix:* guard at the top of every function.
 - 4. Integer overflow in a sum or product.** `int` totals wrap silently and the wrong answer looks plausible. *Fix:* `long long` for totals, `0LL` in `accumulate`, `1LL *` before a multiply.
 - 5. Overflow in the binary-search midpoint.** `(lo + hi) / 2` near `INT_MAX`. *Fix:* `lo + (hi - lo) / 2`, always, even when it cannot overflow — make it a reflex.
 - 6. Mixing the two binary-search templates.** `while (lo < hi)` together with `hi = mid - 1` skips the answer; `while (lo <= hi)` together with `hi = mid` **loops forever**. *Fix:* pick one template from Part 6 and never improvise.
 - 7. Kadane initialised to 0.** Returns 0 on an all-negative array. *Fix:* initialise `best` and `cur` from `v[0]` and start at `i = 1`.
 - 8. Forgetting `seen[0] = 1` in prefix + hash map.** Silently misses every subarray that starts at index 0. *Fix:* seed the map with the empty prefix, and look up **before** you insert.
 - 9. Sliding window used on data with negative numbers.** The shrink step is unjustified and the answer is wrong on mixed-sign input. *Fix:* negatives + sum condition $\Rightarrow$ prefix + hash map.
 - 10. Missing duplicate-skipping in 3Sum.** Duplicate triplets in the output. *Fix:* skip at all three points, always comparing against the element you just left (`v[i] == v[i-1]`, `v[l] == v[l-1]`, `v[r] == v[r+1]`).
 - 11. `mid` advanced after the wrong swap in Dutch flag.** Advancing `mid` after swapping with `high` skips an unexamined value. *Fix:* `mid` moves only in the 0 and 1 branches; loop while `mid <= high`.
 - 12. Forgetting `abs()` when reading a negation-marked array.** By the time you reach `i`, `v[i]` may already be negative, so the index is garbage. *Fix:* `int idx = abs(v[i]) - 1;`.
 - 13. Holding a reference or iterator across a `push_back`.** Reallocation invalidates it, and you write into freed memory. *Fix:* store indices, not references, or `reserve()` up front.
 - 14. Modifying a vector while range-for is iterating it.** `for (int x : v) v.push_back(x);` is undefined behaviour, not an infinite loop you can reason about. *Fix:* iterate by index, or build a separate result vector.
- Two more that are not exactly bugs but cost marks just as reliably: **sorting when the problem asked for indices** (LC 1 with sorting returns positions in the sorted array, which is not the answer), and **`unique()` without `erase()`** (the tail garbage stays and `size()` is unchanged).

PART 12 — Complexity cheat sheet

By pattern

PATTERN	TIME	SPACE	NOTES
Linear scan	$O(n)$	$O(1)$	
Two pointers	$O(n)$	$O(1)$	$O(n \log n)$ if you must sort first
Write index / in-place filter	$O(n)$	$O(1)$	
Dutch flag partition	$O(n)$	$O(1)$	exactly one pass
Sliding window	$O(n)$	$O(1)$ or $O(k)$	each index enters and leaves once
Prefix sum build	$O(n)$	$O(n)$	query $O(1)$
Prefix + hash map	$O(n)$	$O(n)$	average case; $O(n)$ worst on collisions
Difference array	$O(n + q)$	$O(n)$	q range updates
2D prefix sum	$O(m \cdot n)$	$O(m \cdot n)$	query $O(1)$
Binary search	$O(\log n)$	$O(1)$	sorted input required
Binary search on the answer	$O(n \log R)$	$O(1)$	R = size of the answer range
Hash map / set	$O(n)$ avg	$O(n)$	
Cyclic sort / in-place marking	$O(n)$	$O(1)$	needs values in <code>1..n</code>
Sorting	$O(n \log n)$	$O(\log n)$	<code>std::sort</code> is introsort, in place
Matrix traversal	$O(m \cdot n)$	$O(1)$	
Monotonic stack / deque	$O(n)$	$O(n)$	amortised — each index pushed and popped once

The brute force you are replacing

NAIVE	COST	REPLACED BY
All pairs	$O(n^2)$	hash map or two pointers $\rightarrow O(n)$
All triplets	$O(n^3)$	sort + two pointers $\rightarrow O(n^2)$
All subarrays, summing each	$O(n^3)$	prefix sums $\rightarrow O(n^2)$, or window/hash $\rightarrow O(n)$
Re-sum a moving window	$O(n \cdot k)$	rolling window $\rightarrow O(n)$
Linear search, repeated	$O(n \cdot q)$	sort once + binary search $\rightarrow O(n \log n + q \log n)$
Insert into the middle, repeatedly	$O(n^2)$	build then sort, or a difference array

Space you might be spending without noticing

The recursion stack ($O(\text{depth})$), the output array (usually not counted, so "O(1) extra space" still lets you allocate the result), `sort`'s $O(\log n)$ stack, and a `vector<vector<int>>` grid, which costs `m` separate heap allocations rather than one.

PART 13 — The problem list

Thirty-eight problems, ordered so that each one needs only what came before it. Every one of these is solved, tested and explained in the companion book **Arrays-Solved-Questions.pdf** — but attempt each yourself first, for at least twenty-five minutes, before you open it.

Tier 1 — foundations (13 problems)

Do all thirteen before moving on. They are all $O(n)$ one-pass or hash-map problems, and they build the index discipline everything else needs.

#	PROBLEM	DIFFICULTY	PATTERN
1	LC 1 — Two Sum	Easy	Hash map
2	LC 26 — Remove Duplicates from Sorted Array	Easy	Write index
3	LC 27 — Remove Element	Easy	Write index
4	LC 283 — Move Zeroes	Easy	Write index
5	LC 88 — Merge Sorted Array	Easy	Two pointers from the back
6	LC 121 — Best Time to Buy and Sell Stock	Easy	Scan with state
7	LC 53 — Maximum Subarray	Medium	Kadane
8	LC 217 — Contains Duplicate	Easy	Hash set
9	LC 169 — Majority Element	Easy	Boyer-Moore
10	LC 268 — Missing Number	Easy	XOR / sum formula
11	LC 136 — Single Number	Easy	XOR
12	LC 66 — Plus One	Easy	Digit carry
13	LC 448 — Find All Numbers Disappeared	Easy	In-place marking

Tier 2 — the core patterns (15 problems)

This is the tier that decides your placement result. Every one of these is a named pattern, and variations of them are what actually get asked.

#	PROBLEM	DIFFICULTY	PATTERN
14	LC 167 — Two Sum II (sorted)	Medium	Two pointers, opposite ends
15	LC 15 — 3Sum	Medium	Sort + two pointers
16	LC 11 — Container With Most Water	Medium	Two pointers, greedy move
17	LC 42 — Trapping Rain Water	Hard	Two pointers + running max
18	LC 75 — Sort Colors	Medium	Dutch national flag
19	LC 209 — Minimum Size Subarray Sum	Medium	Shortest variable window
20	LC 3 — Longest Substring Without Repeating	Medium	Longest variable window
21	LC 424 — Longest Repeating Character Replacement	Medium	Window + frequency
22	LC 560 — Subarray Sum Equals K	Medium	Prefix + hash map
23	LC 238 — Product of Array Except Self	Medium	Prefix/suffix sweep
24	LC 152 — Maximum Product Subarray	Medium	Kadane with min and max
25	LC 56 — Merge Intervals	Medium	Sort + sweep
26	LC 33 — Search in Rotated Sorted Array	Medium	Binary search
27	LC 153 — Find Minimum in Rotated Sorted Array	Medium	Binary search
28	LC 34 — Find First and Last Position	Medium	lower/upper bound

Tier 3 — the ones that separate candidates (10 problems)

#	PROBLEM	DIFFICULTY	PATTERN
29	LC 41 — First Missing Positive	Hard	Cyclic sort
30	LC 287 — Find the Duplicate Number	Medium	Floyd cycle on an array
31	LC 128 — Longest Consecutive Sequence	Medium	Hash set
32	LC 73 — Set Matrix Zeroes	Medium	Markers in row 0 / column 0
33	LC 48 — Rotate Image	Medium	Transpose + reverse
34	LC 54 — Spiral Matrix	Medium	Four shrinking boundaries
35	LC 189 — Rotate Array	Medium	Three reversals
36	LC 239 — Sliding Window Maximum	Hard	Monotonic deque
37	LC 4 — Median of Two Sorted Arrays	Hard	Binary search on a partition
38	LC 31 — Next Permutation	Medium	Index scan from the right

The Indian-placement classics

These come up in written rounds at TCS, Infosys, Wipro, Cognizant, Accenture and in most college tests, often phrased in words rather than as LeetCode problems. Most are easy once you have Tier 1, and they are worth one focused evening.

- Reverse an array in place; rotate it by k (three-reversal method)
- Find the second largest / second smallest element in **one pass** (watch for duplicates)
- Find the largest and smallest in the minimum number of comparisons (pairwise, $3n/2$ comparisons)
- Left-rotate an array by one, and by d
- Check whether an array is sorted, or sorted and rotated
- Union and intersection of two sorted arrays (two pointers)
- Move all negatives to one side (Dutch-flag two-way partition)
- Find the "leader" elements — every element bigger than everything to its right (scan from the right)
- Equilibrium index — prefix sum from both sides
- Maximum circular subarray sum (total – minimum-subarray, with the all-negative special case)
- Count inversions in an array (merge sort — $O(n \log n)$)
- Find the duplicate and the missing number together ($1..n$, sum and sum-of-squares, or XOR)
- Chocolate distribution problem (sort, then a window of size m , minimise max–min)
- Stock buy and sell to maximise profit with unlimited transactions
- Trapping rain water and largest rectangle in a histogram — always on the list
- Print a matrix in spiral order; rotate it by 90° ; transpose it
- Merge two sorted arrays without extra space (gap method / Shell-sort-style)

GfG's *Top 50 Array Problems* covers all of these; treat it as the written-round syllabus and this guide's Tier 1 and 2 as the interview syllabus.

PART 14 — A four-week plan

Two hours a day. **Attempt first, read the solution second** — the companion book is for after you have struggled, never instead of struggling.

Week 1 — foundations and the index discipline

- Days 1-2: Parts 0-2. Then `practice-01-basics.cpp`, TODOs 1-5. Write out the invariant comment for every loop, even when it feels silly.
- Days 3-5: Tier 1 problems 1-13, roughly three a day.
- Days 6-7: Redo problems 2, 4, 7 and 13 **from a blank file**, without looking. If you cannot, that is the signal to reread, not to move on.

Week 2 — two pointers and windows

- Days 1-2: Parts 3 and 4. Practice TODOs 6-8.
- Days 3-5: Problems 14-21. Before each, say aloud which of the three two-pointer shapes, or which window flavour, it is.
- Days 6-7: Write the variable-window template from memory, then re-solve LC 3 and LC 209 with it.

Week 3 — prefix sums, binary search, hashing

- Days 1-2: Parts 5, 6 and 7. Practice TODOs 9-10.
- Days 3-5: Problems 22-28. Write template 2 of binary search from memory every single day until the boundaries stop needing thought.
- Days 6-7: Binary search on the answer — LC 875 and LC 1011, which are not in the 38 but are the pattern's home ground.

Week 4 — the hard tier and revision

- Days 1-3: Parts 8-12, then problems 29-38.
- Day 4: The Indian-placement classics list, timed, twenty minutes each.
- Days 5-6: A mixed set of twenty problems drawn at random from all three tiers. Random order is the point — pattern recognition is the skill being tested.
- Day 7: Reread Part 11 (the bug catalogue) and the quick-reference card below. Redo any problem you failed during the week.

How to know you are ready: given a new array problem, you can name the pattern within sixty seconds, state the target complexity from the constraints, and write the index invariants before writing any code. Speed comes from recognition, not from typing.

Quick reference card

Before anything: `int n = v.size();` · read the constraints · is it sorted? · can values be negative? · is `n == 0` possible?

```
// two pointers, opposite ends
int l = 0, r = n - 1;
while (l < r) { if (cond) l++; else r--; }

// write index (in-place filter)
int w = 0;
for (int i = 0; i < n; i++) if (keep(v[i])) v[w++] = v[i];
return w;

// variable window: LONGEST
int l = 0;
for (int r = 0; r < n; r++) { add(v[r]);
    while (invalid()) remove(v[l++]);
    best = max(best, r - l + 1); }

// variable window: SHORTEST
int l = 0;
for (int r = 0; r < n; r++) { add(v[r]);
    while (valid()) { best = min(best, r - l + 1); remove(v[l++]); } }

// prefix sums
vector<long long> P(n + 1, 0);
for (int i = 0; i < n; i++)
    P[i+1] = P[i] + v[i];
// sum v[i..j] = P[j+1] - P[i];

// prefix + hash map
unordered_map<long long, int> seen{{0, 1}};
long long pre = 0;
for (int x : v) { pre += x;
    cnt += seen[pre - k]; seen[pre]++; }

// binary search: EXACT
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int m = lo + (hi - lo) / 2;
    if (v[m] == t) return m;
    if (v[m] < t) lo = m + 1;
    else hi = m - 1; }

// binary search: FIRST TRUE (boundary)
int lo = 0, hi = n;
while (lo < hi) {
    int m = lo + (hi - lo) / 2;
    if (pred(m)) hi = m;
    else lo = m + 1;
} return lo;

// Kadane
int best = v[0], cur = v[0];
for (int i = 1; i < n; i++) {
    cur = max(v[i], cur + v[i]);
    best = max(best, cur); }

// Dutch national flag
int low = 0, mid = 0, high = n - 1;
while (mid <= high) {
    if (v[mid] == 0) swap(v[low++], v[mid++]);
    else if (v[mid] == 1) mid++;
    else swap(v[mid], v[high--]); }

// rotate right by k
k %= n;
reverse(v.begin(), v.end());
reverse(v.begin(), v.begin() + k);
reverse(v.begin() + k, v.end());

// cyclic sort (values 1..n)
for (int i = 0; i < n; i++)
    while (v[i] > 0 && v[i] <= n &&
        v[v[i]-1] != v[i])
        swap(v[i], v[v[i]-1]);
```

Pattern triggers: contiguous → window/prefix · sorted → two pointers/binary search · in place, O(1) space → write index, swap, or mark · return indices → hash map · minimise the maximum → binary search on the answer · values in `1..n` → index-as-hash · negatives + sum → prefix + hash map, never a window.

Guards that pay for themselves: `i + 1 < n` · `long long` totals · `lo + (hi-lo)/2` · `seen[0] = 1` · `abs()` when reading marks · `k %= n` · Kadane from `v[0]` · bounds check before access.